



neo4j auradb



All Images Videos Shopping News Maps Web More

Tools

Login Github Tutorial Free Pricing Enterprise

About 22,300 results (1.07 seconds)



Neo4j

https://neo4j.com › cloud › platform › aura-graph-data...

Fully Managed Graph Database Service | Neo4j AuraDB

Neo4j AuraDB logo. The world's leading graph database as a fully-managed cloud service — zero-admin, globally available and always-on.

Neo4j AuraDB

)? Neo4j AuraDB is a highly available DBaaS that requires ...

Neo4j AuraDB Enterprise

Neo4j AuraDB Enterprise empowers you to rapidly deploy ...

Graph Database as a Service

➔ [Get Your Keywords I](#)

neo4j auradb - Google Search x Fully Managed Graph Database x +

https://neo4j.com/cloud/platform/aura-graph-database/

Neo4j-Databricks Connector Delivers Deeper Insights, Faster GenAI Development [Learn More](#)

Aura Login Partners Company Support Q

Products Use Cases Developers & Data Scientists Learn Pricing Contact Us [Get Started Free](#)

THE FASTEST PATH TO GRAPH

neo4j auradb

The world's leading graph database as a fully-managed cloud service — zero-admin, globally available and always-on.

[Start Free](#)

The screenshot displays the Neo4j Workspace interface. On the left, a sidebar titled "Database information" lists various categories like Food, Function, Genre, Image, Issue, Language, Library, Lim model, Market, Media, Model, Movie, Organization, Outcome, Person, Place, Platform, plays, Product, Programming language, Python version, Ranking, Resource, Software, Task, Technique, Tool, and Variable. Below these, a list of relationships (2,719) is shown, including ACTED_IN, ANALYZES, BASICS, BATSMAN_OF, BECAME_TRILLION_DOL..., BESTSELLING, BROWSE, CAN_CREATE, CAN_DO, CHIEF_EXECUTIVE_OFFL..., DESIGNS, DEVELOPS, DIRECTED, EMPTY, FIRST_CHUNK, FOUNDER, GENERATE, GENERATE_CONTENT, GENERATES, HAS, HAS_CHANNEL, HAS_CREATED, HAS_ENTITY, HAS_SHOWN, HAS_TRIED, HEADQUARTERED_IN, IN_GENRE, INCORPORATED, and INSPIRATION.

The main area shows two Cypher queries and their results:

Query 1: `neo4j $ MATCH p=()-[:TRILLION_DOLLAR_COMPANY]->() RETURN p LIMIT 25;`

Results overview for Query 1:

- Nodes (2): `.._Entity_... (2)`, `Company (1)`, `Date (1)`
- Relationships (1): `TRILLION_DOLLAR_COM...`

Query 2: `neo4j $ MATCH p=()-[:CHIEF_EXECUTIVE_OFFICER]->() RETURN p LIMIT 25;`

Results overview for Query 2:

- Nodes (2): `.._Entity_... (2)`, `Company (1)`

IN Graph DB you work with Cypher Query

neo4j

ExploreQueryImport

Instance01 / neo4j

Send feedback

Database information

Nodes (13)

CEO

Company

Country

employee

entrepreneur

Movie

Person

Relationships (4)

ACTED_IN

CEO

LIVES_IN

Property keys

born

name

POB

released

title

YOB

Last update: 1:16:38 PM

neo4j

\$ MATCH p=()-[:ACTED_IN]->() RETURN p LIMIT 25;

Graph

Table

RAW

Download

Copy

Forest Gump

↑ ACTED_IN

Tom Hanks

Results overview

Nodes (2)

Movie (1)

Person (1)

Relationships (1)

ACTED_IN (1)

neo4j

ExploreQueryImport

Instance01 / neo4j

Send feedback

Database information

Nodes (14)

CEO

Company

Country

employee

entrepreneur

Movie

Person

Relationships (5)

ACTED_IN

CEO

LIVES_IN

Property keys

born

name

POB

released

title

YOB

neo4j

\$ MATCH(p:Person {name:'Tom Hanks'}) SET p.born=1957 return p

neo4j

\$ MATCH p=()-[:ACTED_IN]->() RETURN p LIMIT 25;

Graph

Table

RAW

Download

Copy

Krish

→ ACTED_IN

Forest Gump

↑ ACTED_IN

Tom Hanks

Node details

Person

Key	Value
<id>	11
born	1956
name	"Tom Hanks"

The screenshot displays the Neo4j workspace interface. On the left, the 'Database information' panel shows 24 nodes and 5 relationships. The nodes are categorized by type: CEO, Company, Country, employee, entrepreneur, Movie, Person, and User. The relationships are ACTED_IN, CEO, and LIVES_IN. Property keys include born, genre, movieid, name, POB, released, title, and YOB. The main query editor shows a Cypher query: `1 LOAD CSV WITH HEADERS FROM 'https://raw.githubusercontent.com/krishnaik06/graph-dataset/main/users_social.csv';` and `2 .` Below the query, the 'neo4j' instance is selected, and the query is executed. The results are displayed in the 'Graph' view, showing a network of nodes and relationships. The nodes are labeled 'Forest Gump', 'Krish', and 'Tom Hanks'. The relationships are labeled 'ACTED_IN'. The 'Node details' panel on the right shows the details for the 'Person' node, including the key 'id' and the property 'name'.

For example in social media post here is the graph structure, You can load them to tool, by loading them as CSV

```
userId,name,age,city
1,John,30,New York
2,Sarah,25,San Francisco
3,Mike,35,Chicago
4,Linda,28,New York
5,David,32,San Francisco
```

insert this data in neo4j

The screenshot shows the Neo4j workspace interface. On the left, the 'Database information' panel displays nodes and relationships. Nodes (19) include labels like CEO, Company, Country, employee, entrepreneur, Movie, and Person. Relationships (5) include ACTED_IN, CEO, and LIVES_IN. Property keys include born, genre, movieid, name, POB, released, title, and YOB. The main query editor shows a Cypher query:

```
1 LOAD CSV WITH HEADERS FROM 'https://raw.githubusercontent.com/krishnaik06/graph-dataset/main/users_social.csv' AS row
2 CREATE
  (:User{userId:toInteger(row.userId),name:row.name,age:toInteger(row.age),city:row.city});
3
```

 Below the query editor, the execution results are shown. The first query is `$ MATCH (n:User) DELETE n;` which successfully deleted 5 nodes. The second query is `$ MATCH (n:Person) RETURN n LIMIT 25;`.

relations

userId1,userId2,type

1,2,FRIEND

1,3,FRIEND

2,4,FRIEND

2,5,FRIEND

3,4,FRIEND

3,5,FRIEND

4,1,LIKES

4,5,LIKES

1,5,LIKES

5,2,LIKES

2,3,LIKES

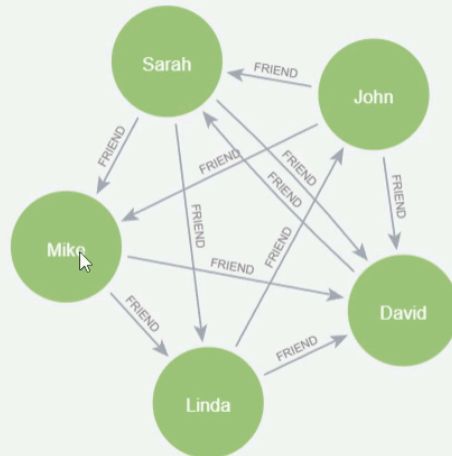
```

1 LOAD CSV WITH HEADERS FROM "https://raw.githubusercontent.com/krishnaik06/graph-dataset/main/relationships.csv" as row
2 MATCH (u1:User {userId: toInteger(row.userId)}), (u2:User {userId: toInteger(row.userId2)})
3 CREATE (u1)-[:FRIEND]->(u2)
4 CREATE (u1)-[:LIKES]->(u2);

```

• neo4j \$ MATCH p=()-[:FRIEND]->() RETURN p LIMIT 25;

Graph Table RAW



activities

postId,content,userId,timestamp

1,Hello World!,1,2024-07-14T12:34:56

2,My first post!,2,2024-07-14T13:00:00

3,Beautiful day in Chicago.,3,2024-07-14T13:45:00

4,Love New York!,4,2024-07-14T14:15:00

5,San Francisco is amazing!,5,2024-07-14T14:30:00

```

1 LOAD CSV WITH HEADERS FROM "https://raw.githubusercontent.com/krishnaik06/graph-dataset/main/posts.csv" as row
2 MATCH (u:User {userId:toInteger(row.userId)})
3 CREATE (u)-[:POSTED]I>(<:Post{postId:toInteger(row.postId),content:row.content,timestamp:datetime(row.timestamp)});

```